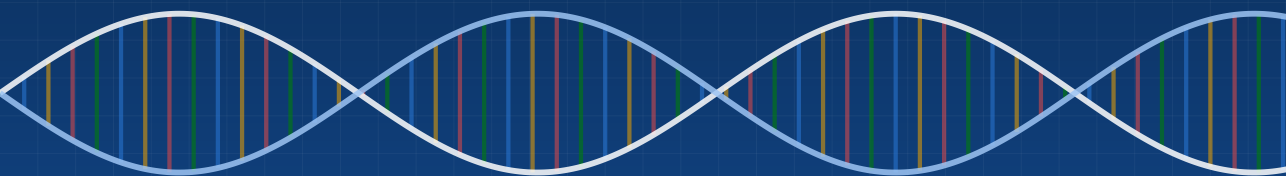


Secuencias, formatos y bases de datos



La buena gestión de los datos no es un fin en sí misma, sino el conducto clave que lleva al descubrimiento y la innovación, y a la posterior integración y reutilización de datos y conocimiento por parte de la comunidad.

— Wilkinson et al. (2016)

● Al terminar este capítulo usted podrá

- Reconocer la pregunta que responde cada formato (FASTA, FASTQ, GenBank, GFF3/GTF, BED, SAM/BAM/CRAM y VCF) y ubicarlo en el flujo de un análisis genómico.
- Convertir entre calidades Phred, probabilidades de error y caracteres ASCII, y calcular los errores esperados de una lectura.
- Traducir sin errores entre coordenadas 1-based cerradas y 0-based semiabiertas, en ambas hebras.
- Interpretar una CIGAR, descomponer una FLAG bit a bit y explicar cómo BGZF y los índices permiten el acceso aleatorio a archivos de cientos de gigabytes.

- Consultar NCBI, Ensembl, UniProt y PDB desde Python mediante APIs REST, respetando límites de tasa, registrando versiones e identificadores estables y aplicando los principios FAIR.

Una secuencia de ADN, por sí sola, es apenas una cadena de letras. Lo que la convierte en un *dato científico* es todo lo que la acompaña: de qué organismo proviene, con qué confianza se leyó cada base, en qué lugar del genoma cae, qué genes la atraviesan y en qué se diferencia de la referencia. Cada una de esas capas de información se guarda en un formato de archivo distinto, y cada formato se deposita en alguna de las grandes bases de datos públicas que, desde los años ochenta, sostienen la biología moderna. Quien no domina ese ecosistema pierde horas (o semanas) persiguiendo errores que no son biológicos sino tipográficos: una coordenada desplazada en una posición, una codificación de calidad mal interpretada, un identificador sin versión.

Este capítulo tiene dos partes. En la primera (sección 2.1) estudiaremos los formatos como lo que realmente son: *contratos* entre programas, con una gramática precisa y, detrás de ella, un poco de matemática (logaritmos para la calidad, álgebra de intervalos para las coordenadas, aritmética binaria para las banderas, un índice jerárquico para el acceso aleatorio). En la segunda (sección 2.2) saldremos a buscar datos reales: seguiremos al gen humano *TP53*, el llamado “guardián del genoma”, a través de NCBI, Ensembl, UniProt y PDB, y aprenderemos a hacerlo de forma reproducible, respetuosa con los servidores y trazable.

2.1 El idioma de los archivos

 Colab · 2.1 · Formatos de archivo

 [abrir notebook](#)

Una cocina profesional

En la cocina de un buen restaurante, la receta está escrita en un cuaderno, los ingredientes llegan en cajas etiquetadas con fecha de vencimiento y grado de frescura, el plan de la noche está en una pizarra y los comentarios de los comensales se anotan en un libro aparte. Nadie mezclaría todo en una sola hoja: cada soporte responde a una pregunta distinta y lo leen personas distintas. Pero todos comparten un vocabulario (el nombre del plato, el número

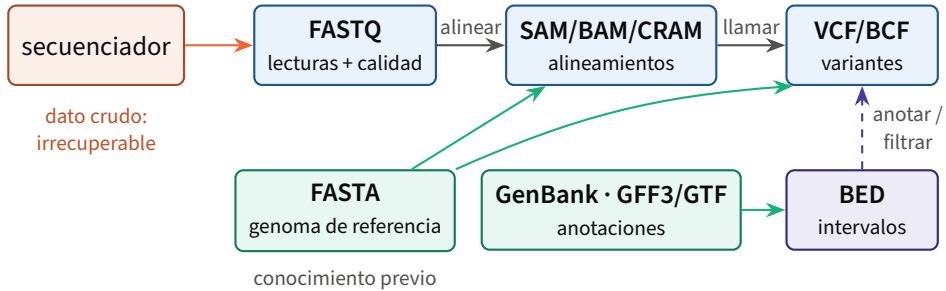


Figura 2.1. La vida de los datos en un análisis de resecuenciación. En azul, los archivos que el propio análisis produce, de izquierda a derecha cada vez más resumidos: de cientos de gigabytes de lecturas a unos pocos megabytes de variantes. En verde, el conocimiento previo (la referencia y sus anotaciones), que se descarga de las bases de datos de la sección 2.2. BED es el formato “comodín” para describir regiones con las que filtrar o anotar. Todo el sistema depende de que los nombres de cromosoma y las coordenadas signifiquen lo mismo en cada archivo.

de mesa) que permite cruzarlos. Los formatos bioinformáticos funcionan igual: uno guarda la receta (la secuencia de referencia), otro los ingredientes con su frescura (las lecturas y su calidad), otro el plan (las anotaciones), otro dónde terminó cada ingrediente (los alineamientos) y otro las quejas (las variantes). El vocabulario común son los nombres de cromosomas y las coordenadas.

2.1.1 Por qué hay tantos formatos

La figura 2.1 resume la vida de los datos en un análisis típico de resecuenciación. El único archivo que “sale de la máquina” es el FASTQ; todo lo demás es el producto de un cálculo (alinear, comparar, anotar) o conocimiento previo (el genoma de referencia y sus genes). De esa observación se desprende una regla práctica: los archivos intermedios se pueden regenerar, los FASTQ crudos no. Por eso se archivan en repositorios públicos como el *Sequence Read Archive* (Leinonen et al., 2011).

Casi todos estos formatos son *de texto delimitado*: una línea por registro, campos separados por tabuladores, encabezados que empiezan con un carácter especial. Esa elección, heredada de la filosofía de Unix, tiene un precio (archivos grandes) y una enorme ventaja: se pueden inspeccionar con `head`, filtrar con `grep` y procesar con `awk` sin ningún programa especializado, como defiende Buffalo (2015). Cuando el tamaño se vuelve inmanejable, cada formato de texto tiene un gemelo binario

comprimido e indexado (BAM para SAM, BCF para VCF), cuya lectura y escritura están implementadas en una biblioteca común, HTSlib (Bonfield et al., 2021).

2.1.2 FASTA: la secuencia desnuda

El formato FASTA nació como el formato de entrada del programa de búsqueda de similitud del mismo nombre (Lipman & Pearson, 1985; Pearson & Lipman, 1988), y sobrevivió al programa porque es casi imposible de simplificar más:

genoma.fasta

```
>NC_045512.2 Severe acute respiratory syndrome coronavirus 2
ATTAAAGGTTTATACCTTCCCAGGTAACAAACCAACCAACTTTCGATCTCTTGATGATCT
GTTCTCTAAACGAACCTTTAAATCTGTGTGGCTGTCACTCGGCTGCATGCTTAGTGCACT
```

Definición 2.1. (Registro FASTA)

Un archivo FASTA es una sucesión de registros. Cada registro comienza con una línea de encabezado que empieza por >; la primera palabra tras el > (hasta el primer espacio) es el *identificador* y el resto es una descripción libre. Las líneas siguientes, hasta el próximo > o el fin del archivo, se concatenan para formar la secuencia, escrita con el alfabeto IUPAC de nucleótidos o de aminoácidos.

Dos detalles de esta definición causan más errores de los que parece. El primero es que muchas herramientas cortan el identificador en el primer espacio: si el encabezado es >chr1 Homo sapiens, el cromosoma se llamará chr1 en el BAM, y un archivo de anotaciones que diga 1 (la convención de Ensembl) no casará con él. El segundo es que el FASTA no tiene un ancho de línea obligatorio, pero los índices (.fai) que permiten extraer una región sin leer el archivo completo sí exigen que todas las líneas de un registro tengan la misma longitud. Con líneas de w caracteres, la base x (0-based) de un registro cuya secuencia empieza en el byte o se encuentra en el byte

$$b(x) = o + \left\lfloor \frac{x}{w} \right\rfloor (w + 1) + (x \bmod w), \quad (2.1)$$

DONDE

$b(x)$	Posición en bytes, dentro del archivo, del carácter que codifica la base x .
o	Desplazamiento (en bytes) donde empieza la primera línea de secuencia del registro; se guarda en el .fa.i.
w	Número de bases por línea; el +1 cuenta el salto de línea \n.
x	Posición de la base dentro de la secuencia, contando desde o .

Es decir: saltamos $\lfloor x/w \rfloor$ líneas completas, cada una de $w + 1$ bytes, y avanzamos $x \bmod w$ caracteres dentro de la última. Si una sola línea tiene otra longitud, la fórmula apunta al lugar equivocado; por eso `samtools faidx` se niega a indexar archivos irregulares. El genoma de SARS-CoV-2 (29 903 nt) escrito con $w = 60$ ocupa $\lceil 29\,903/60 \rceil = 499$ líneas de secuencia.

2.1.3 FASTQ: la secuencia con su nivel de confianza

Un secuenciador no solo dice *qué* base leyó, sino *cuán seguro está*. Esa segunda información es la que nos permite, más adelante, distinguir una mutación real de un error de la máquina. El formato FASTQ, documentado formalmente por Cock et al. (2010) mucho después de volverse ubicuo, guarda cada lectura en exactamente cuatro líneas (figura 2.2): el nombre precedido de @, la secuencia, un separador + y una cadena de calidades con *un carácter por base*.

La escala Phred

La calidad no se guarda como probabilidad, sino en una escala logarítmica introducida por el programa `phred`, que asignaba bases y calidades a las trazas de los secuenciadores capilares del Proyecto Genoma Humano (Ewing & Green, 1998; Ewing et al., 1998).

Definición 2.2. (Calidad Phred)

Si p es la probabilidad de que la base asignada sea incorrecta, su calidad Phred es

$$Q = -10 \log_{10} p \quad \Longleftrightarrow \quad p = 10^{-Q/10}. \quad (2.2)$$

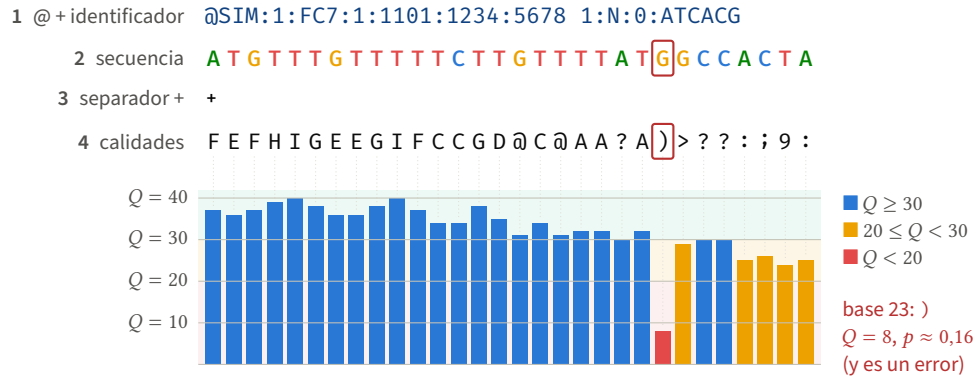


Figura 2.2. Anatomía de un registro FASTQ: las primeras 30 bases del gen S de SARS-CoV-2 leídas por un secuenciador simulado. Cada carácter de la línea 4 codifica la calidad de la base que tiene encima; las barras muestran el valor Phred decodificado ($Q = \text{código ASCII} - 33$). La calidad cae lentamente a lo largo de la lectura, como en un Illumina real. La base 23 tiene $Q = 8$ (carácter `)`: el secuenciador “avisa” que se equivoca con probabilidad $10^{-0,8} \approx 0,16$, y en efecto leyó G donde la referencia tiene T. Generada con `figuras/cap02/generar.py`.

DONDE	
p	Probabilidad estimada de error de la base (entre 0 y 1).
Q	Calidad Phred; se redondea a un entero. Más alta significa más confiable.
-10	Factor de escala: 10 unidades de Q equivalen a un factor 10 en p (como los decibelios).

¿Por qué un logaritmo? Porque lo que importa de una probabilidad de error es su *orden de magnitud*. Pasar de $p = 10^{-2}$ a $p = 10^{-3}$ es una mejora tan importante como pasar de 10^{-3} a 10^{-4} , y la escala Phred convierte ambas en el mismo salto de 10 puntos. Así, $Q = 20$ significa un error en 100 bases, $Q = 30$ uno en mil y $Q = 40$ uno en diez mil. Hay además una razón estadística más profunda: $-\log p$ es la *sorpres*a (en unidades de información) de observar un error, de modo que las calidades de bases independientes se combinan sumando logaritmos, como las puntuaciones de alineamiento del ??.

Ewing y Green (1998) no se limitaron a definir la escala: la *calibraron*. Compararon millones de bases asignadas por *phred* con secuencias de referencia de alta calidad y verificaron que, entre todas las bases con calidad Q , la fracción de errores observada

era, en efecto, cercana a $10^{-Q/10}$. Esta calibración es la que da sentido a todo lo que sigue: una calidad Phred no es un “puntaje” arbitrario, sino una predicción probabilística verificable.

Solexa: la otra escala.. Los primeros secuenciadores Solexa (luego Illumina) usaron una escala distinta, basada en la razón de probabilidades (*odds*) en lugar de la probabilidad (Cock et al., 2010):

$$Q_{\text{Sol}} = -10 \log_{10} \frac{p}{1-p}. \quad (2.3)$$

donde Q_{Sol} puede ser negativa (hasta -5 en la práctica). Despejando p en ambas definiciones y eliminándola se obtiene la conversión exacta

$$Q_{\text{Phred}} = 10 \log_{10} (10^{Q_{\text{Sol}}/10} + 1). \quad (2.4)$$

DONDE

Q_{Sol} Calidad Solexa; $p/(1-p)$ es la razón de probabilidades de error frente a acierto (*odds*).

Q_{Phred} Calidad Phred equivalente (ecuación (2.2)); siempre ≥ 0 .

Para calidades altas, $p \ll 1$ y $p/(1-p) \approx p$, de modo que ambas escalas coinciden: $Q_{\text{Phred}} = 20$ corresponde a $Q_{\text{Sol}} = 10 \log_{10} 99 = 19,96$. Solo divergen cuando la base es muy dudosa (figura 2.3): $Q_{\text{Phred}} = 3$ equivale a $Q_{\text{Sol}} \approx 0$, y el mínimo Solexa, -5 , corresponde a $p = 10^{0,5}/(1 + 10^{0,5}) \approx 0,76$, una base peor que una elección al azar entre cuatro.

De número a carácter

Para que la línea de calidades tenga la misma longitud que la secuencia, cada Q se escribe como un único carácter ASCII. Los primeros 32 códigos ASCII son invisibles (tabuladores, saltos de línea), así que se suma un desplazamiento: el estándar actual, heredado del Instituto Sanger, es *Phred*+33, de modo que $Q = 0$ se escribe ! (código 33), $Q = 30$ se escribe ? (63) y $Q = 40$ se escribe I (73). La figura 2.4 muestra la tabla completa para el rango usual.

Adivinar la codificación

El FASTQ no declara su codificación. Los archivos de Illumina anteriores a 2011 (versiones 1.3 a 1.7 de su programa) usan *Phred*+64, y todavía circulan en

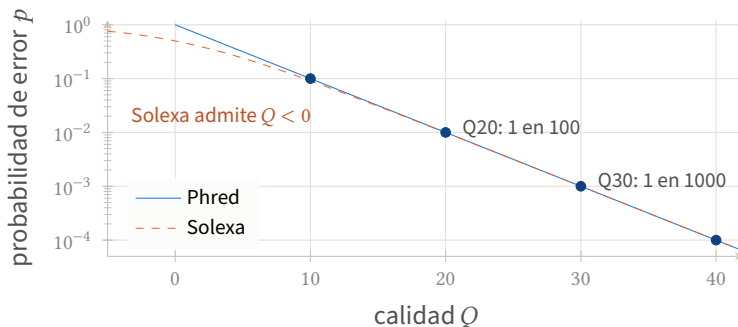


Figura 2.3. La ecuación (2.2) es una recta en escala logarítmica; cada 10 puntos de Q la probabilidad de error se divide entre 10. La escala Solexa (ecuación (2.3)) se superpone a Phred para calidades altas ($Q \geq 15$) y solo diverge cuando p se acerca a 1, donde admite valores negativos. Por eso confundir las escalas apenas afecta a las bases buenas, pero sí a las dudosas, precisamente las que un filtro de calidad debe descartar.

repositorios. Si se lee un archivo Phred+64 como Phred+33, todas las calidades aparecen infladas en 31 puntos: una base con $p = 0,1$ parece tener $p \approx 10^{-4}$, y ningún filtro de calidad la eliminará. Antes de analizar datos antiguos, compruebe el rango de caracteres presentes (figura 2.4); herramientas como FastQC lo hacen automáticamente.

Errores esperados de una lectura

¿Cuántos errores tiene una lectura? No lo sabemos, pero podemos calcular cuántos *esperamos*. Sea X_i la variable indicadora de que la base i es errónea; entonces $E[X_i] = p_i$ y, por la linealidad de la esperanza (que no exige independencia),

$$EE = E\left[\sum_{i=1}^{\ell} X_i\right] = \sum_{i=1}^{\ell} p_i = \sum_{i=1}^{\ell} 10^{-Q_i/10}. \quad (2.5)$$

Si además suponemos que los errores ocurren de forma independiente, la probabilidad de que la lectura esté completamente libre de errores es

$$\mathbb{P}(\text{sin errores}) = \prod_{i=1}^{\ell} (1 - p_i) \approx e^{-EE}, \quad (2.6)$$

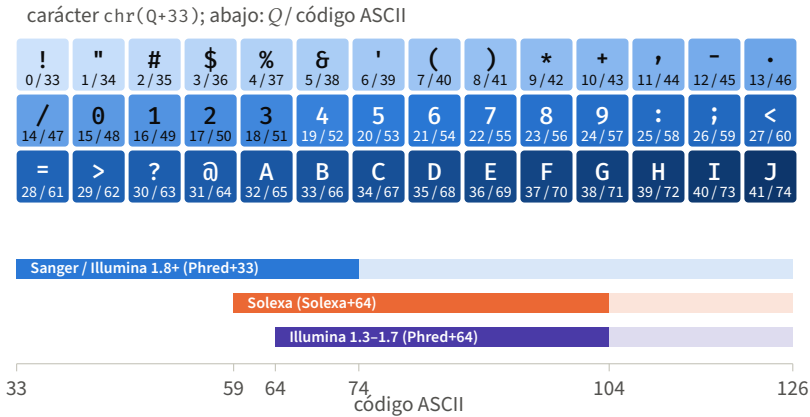


Figura 2.4. Arriba: la codificación Phred+33, carácter por carácter, coloreada desde calidades bajas (claro) hasta altas (oscuro). Abajo: los tres “dialectos” históricos de FASTQ (Cock et al., 2010). La banda clara es el rango permitido por cada codificación; la banda oscura, el rango que aparece en la práctica. Los rangos típicos se solapan entre los códigos 64 y 74 (@–J): un archivo cuyas calidades caen todas ahí es ambiguo, pero basta un solo carácter por debajo de ; (código 59) para concluir que es Phred+33.

DONDE

ℓ	Longitud de la lectura (número de bases).
X_i	Indicadora del evento “la base i es incorrecta” (vale 0 o 1).
p_i, Q_i	Probabilidad de error y calidad Phred de la base i .
EE	Errores esperados (<i>expected errors</i>) de la lectura.

donde la aproximación usa $\ln(1 - p) \approx -p$ para p pequeño: el número de errores es aproximadamente una variable de Poisson de media EE. Esta cantidad resume una lectura mucho mejor que la calidad *media*, porque la media de Q es la media de logaritmos y esconde las pocas bases malas que dominan la suma de probabilidades.

Ejemplo 2.1. (Errores esperados, a mano y en la figura 2.2)

Una lectura corta tiene calidades $Q = (40, 30, 20, 20, 10)$. Sus probabilidades de error son $(10^{-4}, 10^{-3}, 10^{-2}, 10^{-2}, 10^{-1})$ y

$$EE = 0,0001 + 0,001 + 0,01 + 0,01 + 0,1 = 0,1211.$$

Casi todo el valor procede de la única base con $Q = 10$. Su calidad media es 24, que “parece” buena, y sin embargo la probabilidad de que la lectura sea perfecta es solo $\prod(1 - p_i) = 0,8811$ (la aproximación $e^{-0,1211} = 0,8859$ es buena).

En la lectura de 30 bases de la figura 2.2 la calidad media es 32,5, pero $EE = 0,183$ y $P(\text{sin errores}) = 0,821$: la base 23 ($Q = 8$, $p = 0,158$) aporta por sí sola el 87 % de los errores esperados.

La figura 2.5 aplica estas ideas a una corrida simulada de 4000 lecturas de 150 nt, generada con el mismo modelo del notebook: una calidad media que cae de 36,9 en el primer ciclo a 24,1 en el último, más un 6 % de *clusters* defectuosos cuya calidad se desploma en la cola. Los errores se introdujeron exactamente con la probabilidad que dicta su calidad, así que conocemos la verdad: la tasa real de error resultó ser de 0,20 % (0,304 errores por lectura) y la media de EE, 0,296. El panel derecho muestra que los errores esperados predicen los observados, lo que solo ocurre si las calidades están bien calibradas. En datos reales no siempre es así, y por eso existen métodos de recalibración de calidades que estudiaremos junto con la llamada de variantes.

```
fastq_ee.py

import math
from Bio import SeqIO                                     # Biopython >= 1.83

def expected_errors(quals):
    return sum(10 ** (-q / 10) for q in quals)

kept = []
for rec in SeqIO.parse("lecturas.fastq", "fastq"): # Phred+33
    q = rec.letter_annotations["phred_quality"]
    ee = expected_errors(q)
    if ee <= 1.0:                                         # filtro por EE
        kept.append(rec)
    p0 = math.prod(1 - 10 ** (-x / 10) for x in q)
print(len(kept), "lecturas con EE <= 1")
SeqIO.write(kept, "filtradas.fastq", "fastq")
```

Biopython (Cock et al., 2009) decodifica las calidades según el nombre del formato: "fastq" (o "fastq-sanger") para Phred+33, "fastq-illumina" para Phred+64 y "fastq-solexa" para Solexa+64, y convierte entre ellos con la ecuación (2.4) al escribir.

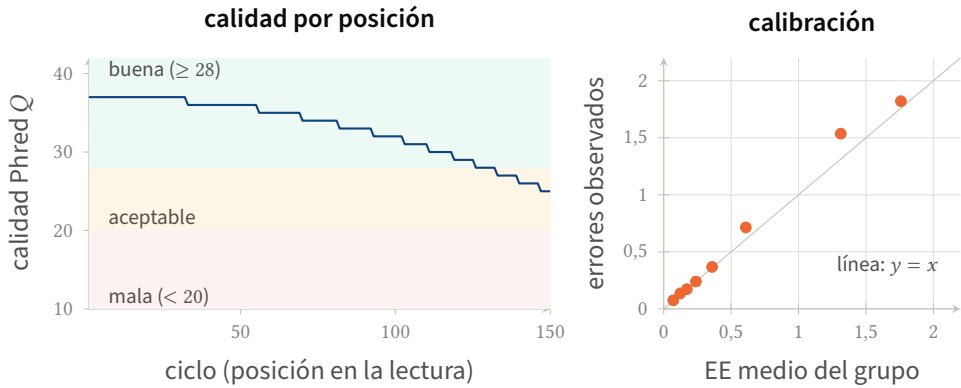


Figura 2.5. Izquierda: calidad por ciclo de 4000 lecturas simuladas, al estilo de FastQC. La línea es la mediana; la banda oscura contiene el 50 % central de las lecturas y la clara el 80 %. La caída hacia el final refleja el desfase progresivo de las moléculas dentro de cada *cluster*; el ensanchamiento de la banda inferior se debe a los *clusters* defectuosos. Esta forma es la que justifica recortar las colas antes de alinear. Derecha: las lecturas se agrupan por su EE (ecuación (2.5)) y se compara el EE medio de cada grupo con el número medio de errores realmente introducidos. Los puntos caen sobre la diagonal: con calidades calibradas, EE es un predictor insesgado del número de errores.

2.1.4 GenBank y el sistema INSDC

Un FASTA dice *qué* es la secuencia; un registro GenBank dice además *qué hay en ella*. El formato, con su encabezado (LOCUS, DEFINITION, ACCESSION, VERSION, SOURCE, referencias bibliográficas), su tabla de características (FEATURES: genes, CDS, UTR, péptidos maduros, con sus coordenadas y calificadores) y la secuencia al final (ORIGIN), es el formato nativo de GenBank, la base de datos pública de secuencias del NCBI (Sayers, Cavanaugh et al., 2025). GenBank intercambia sus datos diariamente con el *European Nucleotide Archive* (ENA) y con el *DNA Data Bank of Japan* (DDBJ): las tres forman la *International Nucleotide Sequence Database Collaboration* (INSDC), de modo que una secuencia depositada en cualquiera de ellas aparece en las tres con el mismo número de acceso (Arita et al., 2021).

fragmento de NC_045512.2.gb

```
gene          21563..25384
              /gene="S"
CDS           21563..25384
```

```

/gene="S"
/codon_start=1
/product="surface glycoprotein"
/protein_id="YP_009724390.1"

```

Las coordenadas de GenBank son *1-based e inclusivas*: el gen S ocupa las bases 21 563 a 25 384, ambas incluidas, es decir, $25\,384 - 21\,563 + 1 = 3822$ nucleótidos (1273 codones más el de terminación). Las localizaciones admiten una pequeña gramática: `complement(100..200)` para la hebra negativa, `join(1..50,80..120)` para exones separados, `<1..300` para un extremo incompleto. Biopython la traduce a objetos `SimpleLocation` y `CompoundLocation`, que, como veremos enseguida, usan internamente coordenadas *o-based*.

2.1.5 Coordenadas: el error de uno

Aquí está la fuente de errores más famosa de la bioinformática. Existen dos convenciones para numerar posiciones en una secuencia, y ambas son correctas a su manera (figura 2.6).

Definición 2.3. (Sistemas de coordenadas)

En el sistema *1-based cerrado*, las posiciones numeran *bases*: la primera base es la 1 y un intervalo $[s_1, e_1]$ incluye ambos extremos. Lo usan GenBank, GFF3/GTF, SAM (columna POS), VCF y los navegadores genómicos. En el sistema *o-based semiabierto*, las posiciones numeran los *bordes entre bases*, como las marcas de una regla: el primer borde es el 0 y un intervalo $[s_0, e_0]$ va del borde s_0 al borde e_0 , incluyendo la base que sigue a s_0 pero no la que sigue a e_0 . Lo usan BED, el formato binario BAM, Python (`seq[s:e]`) y la mayoría de las bibliotecas.

La conversión entre ambos sistemas se deduce de la figura. La base número s_1 (1-based) está a la derecha del borde $s_1 - 1$, y la última base del intervalo, e_1 , termina en el borde e_1 :

$$s_0 = s_1 - 1, \quad e_0 = e_1, \quad \text{longitud} = e_0 - s_0 = e_1 - s_1 + 1. \quad (2.7)$$

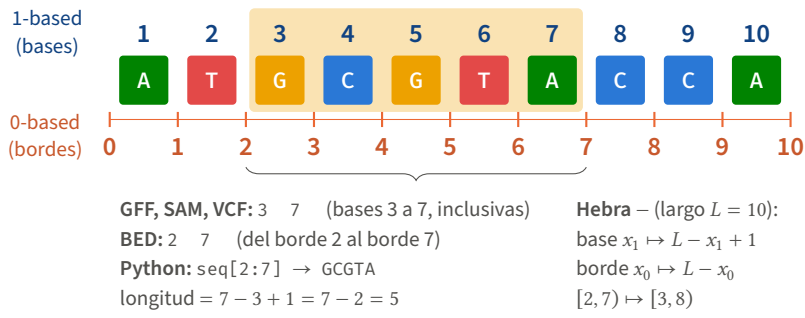


Figura 2.6. Las dos convenciones de coordenadas sobre la misma secuencia. Arriba, en azul, se numeran las bases (1-based); abajo, en naranja, los bordes entre bases, como en una regla (0-based). El intervalo resaltado, GCGTA, se escribe 3-7 en GFF y 2-7 en BED: el inicio cambia en una unidad, el final no. A la derecha, la regla para pasar a la hebra complementaria: en el sistema de bordes basta reflejar ($x_0 \mapsto L - x_0$) e intercambiar los extremos.

DONDE	
s_1, e_1	Primera y última base del intervalo en el sistema 1-based cerrado (GFF, VCF, SAM).
s_0, e_0	Bordes inicial y final del intervalo en el sistema 0-based semiabierto (BED, Python).

¿Por qué los programadores prefieren el sistema semiabierto? Porque su álgebra es más limpia. La longitud es una resta directa, sin el +1 que tantas veces se olvida; dos intervalos contiguos comparten el borde ($[0, 5]$ y $[5, 10]$) ni se solapan ni dejan hueco; un intervalo vacío se representa naturalmente como $[s, s]$, lo que permite describir el *punto* donde ocurre una inserción; y la condición de solapamiento es simétrica:

$$[a, b] \cap [c, d] \neq \emptyset \iff \max(a, c) < \min(b, d), \tag{2.8}$$

$$|[a, b] \cap [c, d]| = \max(0, \min(b, d) - \max(a, c)). \tag{2.9}$$

DONDE	
$[a, b], [c, d]$	Dos intervalos semiabiertos en el mismo cromosoma.
$ \cdot $	Longitud (número de bases) de la intersección.

Esta es la operación elemental de herramientas como BEDTools (Quinlan & Hall, 2010), que responden preguntas como “¿qué variantes caen en exones?” o “¿qué

picos de ChIP-seq solapan promotores?”. Con un índice ordenado por inicio, la búsqueda de todos los intervalos que solapan una consulta cuesta $O(\log n + k)$ en lugar de $O(n)$, donde k es el número de resultados.

Finalmente, la hebra negativa. Si una secuencia tiene longitud L , el borde x_0 de la hebra positiva corresponde al borde $L - x_0$ de su reverso complementario, de modo que $[s_0, e_0) \mapsto [L - e_0, L - s_0)$. En el sistema 1-based, la base x_1 corresponde a la base $L - x_1 + 1$. Los formatos de anotación *no* aplican esa transformación: GFF, BED y VCF expresan siempre las coordenadas sobre la hebra positiva y guardan la hebra en una columna aparte (+/-). Para obtener la secuencia de un gen en la hebra negativa se extrae el intervalo de la hebra positiva y luego se toma su reverso complementario.

Ejemplo 2.2. (El gen S en tres sistemas)

GenBank dice que el gen S de SARS-CoV-2 ocupa 21563..25384. En BED se escribe NC_045512.2 21562 25384, y en Python la secuencia es `ref[21562:25384]`, cuyas tres primeras letras son **ATG**. Si por descuido escribimos `ref[21563:25384]`, obtenemos una secuencia que empieza en **TGT**: el marco de lectura se desplaza y la traducción es basura. La longitud es $25384 - 21562 = 3822$ en ambos sistemas.

En la otra dirección, el servicio esummary del NCBI informa para *TP53* humano (hebra negativa del cromosoma 17) `chrstart = 7687489` y `chrstop = 7668420`, en coordenadas 0-based y en el sentido de lectura del gen. Sumando 1 a cada extremo y ordenándolos se obtiene 7 668 421–7 687 490, que es exactamente lo que reporta Ensembl, en 1-based, para el transcrito canónico (sección 2.2).

⚠ Tres síntomas del error de uno

1. *Traducciones sin sentido*: una proteína llena de codones de parada suele indicar un marco desplazado en una base.
2. *Alelos de referencia que no coinciden*: en un VCF correcto, la columna REF debe ser igual a la base de la referencia en POS. Si casi ninguna coincide, alguien convirtió mal las coordenadas. Es la prueba más barata y eficaz que existe.

3. *Intervalos de longitud cero o negativa*: un BED con `start = end` tras una conversión suele venir de restar 1 al final en lugar de al inicio.

2.1.6 GFF3, GTF y BED: anotaciones e intervalos

El GenBank es riquísimo, pero su estructura depende de sangrías y es incómodo de procesar. Para intercambiar anotaciones se usa el *General Feature Format*, cuya versión 3 (GFF3) es un archivo tabular de nueve columnas: `seqid`, `source`, `type`, `start`, `end`, `score`, `strand`, `phase` y `attributes`. La columna `type` no es texto libre: debe ser un término de la *Sequence Ontology* (`gene`, `mRNA`, `exon`, `CDS`, `five_prime_UTR`...), un vocabulario controlado creado precisamente para que las anotaciones de distintos proyectos significaran lo mismo (Eilbeck et al., 2005). Los atributos ID y Parent enlazan las filas y forman un árbol: un gen es padre de sus transcritos, cada transcrito es padre de sus exones y CDS.

sars.gff3

```
##gff-version 3
NC_045512.2 RefSeq gene 21563 25384 . + . ID=gene-S;Name=S
NC_045512.2 RefSeq CDS 21563 25384 . + 0 ID=cds-S;Parent=gene-S
```

El GTF (*Gene Transfer Format*, a veces llamado GFF2.5) usa las mismas nueve columnas pero con atributos obligatorios `gene_id` y `transcript_id` y una sintaxis distinta (`gene_id "TP53";`). Ensembl y GENCODE lo distribuyen porque es la entrada estándar de los programas de cuantificación de RNA-seq.

La columna `phase` merece una explicación, porque es la menos intuitiva. Un CDS partido en exones no tiene por qué empezar cada fragmento al inicio de un codón. La fase de un fragmento es el número de bases que hay que saltar desde su inicio para llegar al primer codón completo. Si antes de ese fragmento ya se recorrieron d bases codificantes,

$$\phi = (3 - d \bmod 3) \bmod 3 \in \{0, 1, 2\}. \quad (2.10)$$

DONDE

ϕ	Fase del fragmento de CDS (columna 8 del GFF3).
d	Número de bases codificantes contenidas en los fragmentos anteriores, en el sentido de la traducción.

Por ejemplo, si el primer exón codificante tiene 100 bases, $d = 100$, $100 \bmod 3 = 1$ y el segundo fragmento tiene fase $\phi = 2$: sus dos primeras bases completan el codón que quedó abierto.

El formato BED, creado para el navegador genómico de la Universidad de California en Santa Cruz (Kent et al., 2002), es aún más sencillo: tres columnas obligatorias (chrom, chromStart, chromEnd) en coordenadas o-based semiabiertas, y hasta nueve opcionales (nombre, puntuación, hebra, colores, bloques). Es el formato de las *regiones*: exones de un panel de captura, picos de ChIP-seq, regiones de baja complejidad que se deben excluir. Convertir de GFF3 a BED consiste, sencillamente, en aplicar la ecuación (2.7): restar 1 al inicio y dejar el final.

2.1.7 SAM, BAM y CRAM: dónde cayó cada lectura

Tras la secuenciación, un alineador (??) busca en qué lugar de la referencia encaja cada lectura. El resultado se escribe en el formato *Sequence Alignment/Map* (SAM), propuesto por Li et al. (2009) en el contexto del Proyecto 1000 Genomas para poner fin a la proliferación de formatos propios de cada alineador. Un SAM tiene un encabezado (líneas que empiezan con @: @SQ para cada secuencia de referencia con su longitud, @RG para los grupos de lecturas, @PG para los programas que lo procesaron) y una fila por alineamiento con once campos obligatorios, resumidos en la tabla 2.1.

La columna MAPQ reutiliza la escala Phred para otra probabilidad, la de que la *posición* sea incorrecta: $\text{MAPQ} = -10 \log_{10} P(\text{posición errónea})$. Un MAPQ de 60 corresponde a una probabilidad de error de 10^{-6} ; un MAPQ de 0 significa que la lectura encaja igual de bien en al menos otro lugar del genoma (una repetición), y no debe usarse para llamar variantes. Cada alineador estima esta probabilidad a su manera, por lo que los valores de MAPQ no son comparables entre programas.

La CIGAR: instrucciones para colocar la lectura

La CIGAR (*Compact Idiosyncratic Gapped Alignment Report*) codifica el alineamiento como una sucesión de pares (n_k, o_k) : una longitud y una operación. La clave para leerla es saber qué operaciones *consumen* bases de la lectura, de la referencia o de ambas (tabla 2.2).

Tabla 2.1. Los once campos obligatorios de una línea de alineamiento SAM.

#	Campo	Tipo	Ejemplo	Significado
1	QNAME	texto	r2	nombre de la lectura (el mismo para las dos del par)
2	FLAG	entero	99	doce banderas codificadas en bits (figura 2.8)
3	RNAME	texto	NC_045512.2	secuencia de referencia
4	POS	entero	21608	primera base alineada, <i>1-based</i>
5	MAPQ	entero	60	confianza en la posición, en escala Phred
6	CIGAR	texto	3S9M2I8M	cómo encaja la lectura (figura 2.7)
7-9	RNEXT, PNEXT, TLEN		= 21790 332	posición de la pareja y longitud del fragmento
10	SEQ	texto	TGTTT...	bases, en la orientación de la referencia
11	QUAL	texto	FEFHI...	calidades Phred+33

Teorema 2.1. (Longitudes implícitas en una CIGAR)

Sea una CIGAR $C = (n_1, o_1) \cdots (n_K, o_K)$. La longitud de la lectura almacenada en SEQ, el tramo de referencia cubierto y la última posición alineada son

$$\ell_q = \sum_{k: o_k \in \{M, I, S, =, X\}} n_k, \quad (2.11)$$

$$\ell_r = \sum_{k: o_k \in \{M, D, N, =, X\}} n_k, \quad (2.12)$$

$$\text{END} = \text{POS} + \ell_r - 1. \quad (2.13)$$

Tabla 2.2. Operaciones de la CIGAR y qué secuencia consume cada una.

Op.	Nombre	Lect.	Ref.	Uso típico
M	alineada (coincida o no)	sí	sí	base frente a base
= / X	coincidencia / sustitución	sí	sí	versión explícita de M
I	inserción	sí	no	bases que la referencia no tiene
D	delección	no	sí	bases de la referencia ausentes en la lectura
N	salto	no	sí	intrón en RNA-seq
S	recorte suave	sí	no	extremo no alineado, presente en SEQ
H	recorte duro	no	no	extremo no alineado, ausente de SEQ

DONDE

n_k, o_k	Longitud y operación del k -ésimo elemento de la CIGAR.
ℓ_q	Número de bases de la lectura guardadas en SEQ; debe coincidir con la longitud de SEQ.
ℓ_r	Número de bases de la referencia que abarca el alineamiento.
POS, END	Primera y última posición de la referencia cubiertas (1-based, inclusivas).

La demostración es inmediata: al recorrer la CIGAR, un puntero avanza sobre la lectura y otro sobre la referencia, cada uno según la tabla. Lo importante es la consecuencia práctica: los recortes suaves *no* desplazan POS, que siempre apunta a la primera base *alineada*. En o-based semiabierto, el alineamiento ocupa $[\text{POS} - 1, \text{POS} - 1 + \ell_r]$, que es lo que devuelve `pysam` como `reference_start` y `reference_end`.

Ejemplo 2.3. (Leer una CIGAR)

Una lectura tiene $\text{POS} = 1000$ y $\text{CIGAR} = 3\text{S}40\text{M}1\text{I}20\text{M}5\text{D}30\text{M}2\text{S}$. Según las ecuaciones (2.11) y (2.12),

$$\ell_q = 3 + 40 + 1 + 20 + 30 + 2 = 96, \quad \ell_r = 40 + 20 + 5 + 30 = 95,$$

de modo que SEQ debe tener 96 caracteres y el alineamiento termina en $\text{END} = 1000 + 95 - 1 = 1094$. Las bases 1 a 3 de la lectura (recorte suave) no están alineadas; la base 4 de la lectura está frente a la posición 1000 de la

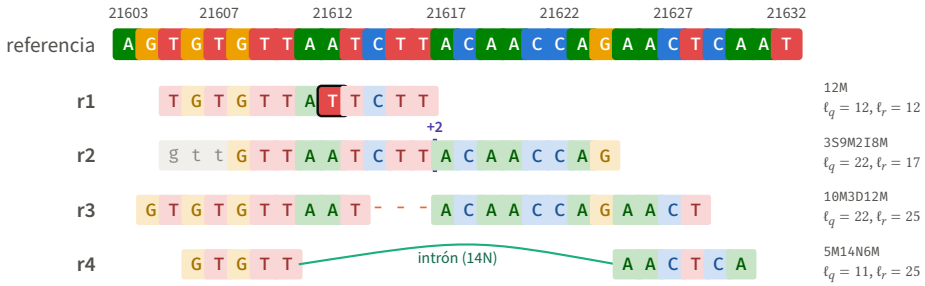


Figura 2.7. Cuatro lecturas alineadas contra el inicio del gen S de SARS-CoV-2 (posiciones 1-based arriba). **r1**: coincidencia completa con una sustitución (recuadro), que la CIGAR M no distingue. **r2**: tres bases recortadas (en minúscula, a la izquierda de POS), luego una inserción de dos bases, marcada entre dos posiciones porque no ocupa lugar en la referencia; la lectura mide 22 bases pero cubre solo 17 de la referencia. **r3**: una delección de tres bases, que sí ocupa referencia. **r4**: una lectura de RNA-seq que salta un intrón de 14 bases (N). A la derecha, ℓ_q y ℓ_r según las ecuaciones (2.11) y (2.12).

referencia; la inserción aparece entre las posiciones 1039 y 1040; la delección elimina las posiciones 1060 a 1064.

La FLAG: doce preguntas en un número

La columna FLAG guarda doce respuestas de sí o no en un único entero. Cada respuesta es un bit $b_k \in \{0, 1\}$ y el número es

$$F = \sum_{k=0}^{11} b_k 2^k, \quad b_k = \left\lfloor F/2^k \right\rfloor \text{ mód } 2. \quad (2.14)$$

DONDE

F	Valor de la columna FLAG (entero entre 0 y 4095).
b_k	Bit k : 1 si la condición k se cumple (p. ej., b_4 : la lectura está en la hebra reversa).
2^k	Valor decimal del bit k : 1, 2, 4, ..., 2048.

Como la representación binaria de un entero es única, la descomposición también lo es: cada FLAG corresponde exactamente a una combinación de condiciones. En código, la pregunta “¿está encendido el bit de valor 16?” se formula con el Y binario: `flag & 16 != 0`. La figura 2.8 muestra las banderas más frecuentes.

	suplement. 2048	duplicado 1024	falla QC 512	secundaria 256	R2 128	R1 64	pareja – 32	hebra – 16	pareja no al. 8	no alineada 4	par correcto 2	pareada 1	
99	0	0	0	0	0	1	1	0	0	0	1	1	1 + 2 + 32 + 64 R1 de un par típico
147	0	0	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 R2 de ese par
83	0	0	0	0	0	1	0	1	0	0	1	1	1 + 2 + 16 + 64 R1 en hebra –
163	0	0	0	0	1	0	1	0	0	0	1	1	1 + 2 + 32 + 128 R2 en hebra +
4	0	0	0	0	0	0	0	0	0	1	0	0	4 no alineada
1171	0	1	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 + 1024 R2 duplicado
2064	1	0	0	0	0	0	0	1	0	0	0	0	16 + 2048 suplementaria, hebra –

Figura 2.8. Siete valores de FLAG frecuentes, descompuestos en sus doce bits (el de menor valor a la derecha, como en la escritura binaria). 99 y 147 son la pareja canónica de un par bien alineado: R1 en la hebra directa con su pareja en la reversa, y viceversa. 83/163 es la configuración espejo. 4 es una lectura sin alinear. 1171 es la R2 anterior marcada como duplicado de PCR. 2064 es un alineamiento suplementario: un trozo de una lectura quimérica que cae en otro lugar del genoma, señal típica de reordenamientos estructurales.

Ejemplo 2.4. (Construir y descomponer una FLAG)

¿Qué FLAG tiene la segunda lectura de un par, bien emparejada, alineada en la hebra reversa y marcada como duplicado? Encendemos los bits de valor 1 (pareada), 2 (par correcto), 16 (hebra reversa), 128 (R2) y 1024 (duplicado): $F = 1 + 2 + 16 + 128 + 1024 = 1171$.

A la inversa, $F = 83$: la mayor potencia de 2 que cabe es 64 (resta 19), luego 16 (resta 3), luego 2 (resta 1) y 1. Así, $83 = 64 + 16 + 2 + 1$: primera lectura del par, en la hebra reversa, par correcto. Su pareja típica es $163 = 128 + 32 + 2 + 1$.

BAM, BGZF e índices: comprimir sin perder el acceso

Un SAM de un genoma humano a 30× pesa cientos de gigabytes. El formato BAM, su versión binaria, resuelve dos problemas a la vez. El primero es el tamaño: los campos numéricos se guardan como enteros binarios, las bases se empaquetan en 4 bits y el resultado se comprime. El segundo, más sutil, es el *acceso aleatorio*. Un archivo gzip ordinario es un único flujo comprimido: para leer el último megabyte hay que descomprimir todo lo anterior. BAM usa en cambio BGZF (*Blocked GNU Zip*

Format): una concatenación de bloques gzip independientes, cada uno con a lo sumo 64 KiB de datos sin comprimir (Li et al., 2009). Cualquier programa que entienda gzip puede leerlo, pero uno que conozca la estructura puede saltar directamente a un bloque.

Para ello, cada posición del archivo se describe con un *desplazamiento virtual* de 64 bits:

$$v = c \cdot 2^{16} + u, \quad 0 \leq u < 2^{16}, \quad (2.15)$$

DONDE

v	Desplazamiento virtual, el número que se guarda en el índice.
c	Desplazamiento en bytes, dentro del archivo <i>comprimido</i> , del inicio del bloque BGZF (48 bits).
u	Desplazamiento dentro del bloque, una vez <i>descomprimido</i> (16 bits, pues el bloque mide a lo sumo 64 KiB).

Leer en v significa: saltar al byte c , descomprimir un solo bloque y avanzar u bytes. Como los desplazamientos virtuales conservan el orden del archivo, se pueden comparar como enteros ordinarios.

Falta saber *qué* v corresponde a una región del genoma. Si el BAM está ordenado por coordenada, el índice `.bai` lo responde con un esquema de *casillas* (*bins*) heredado del navegador de UCSC (Kent et al., 2002): el cromosoma se divide en una jerarquía de seis niveles, y cada alineamiento se asigna a la casilla más pequeña que lo contiene por completo (figura 2.9).

Teorema 2.2. (*Esquema de casillas del índice BAI*)

En el nivel $l \in \{0, \dots, 5\}$ hay 8^l casillas, cada una de 2^{29-3l} pares de bases, numeradas consecutivamente a partir de

$$\text{off}(l) = \frac{8^l - 1}{7} \in \{0, 1, 9, 73, 585, 4681\}. \quad (2.16)$$

Un intervalo $[s, e)$ se asigna a la casilla $\text{off}(l^*) + \lfloor s/2^{29-3l^*} \rfloor$, donde l^* es el nivel más profundo en el que $\lfloor s/2^{29-3l} \rfloor = \lfloor (e-1)/2^{29-3l} \rfloor$. En total hay $\sum_{l=0}^5 8^l = 37\,449$ casillas.

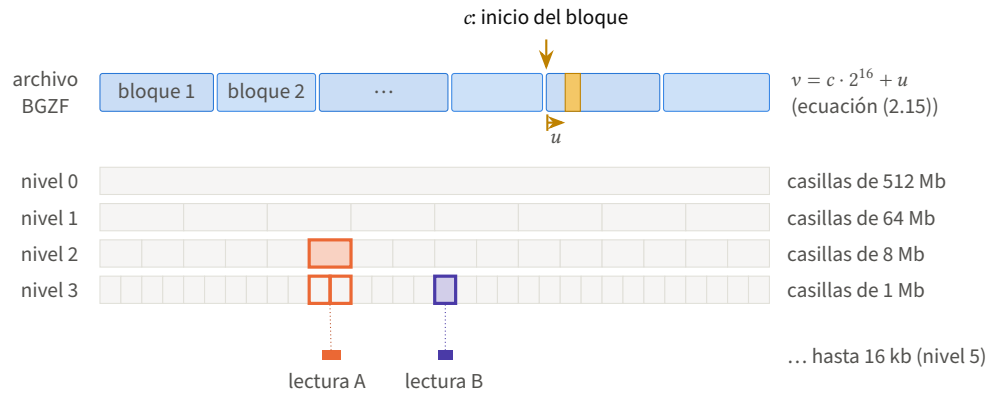


Figura 2.9. Arriba: un archivo BGZF es una cadena de bloques gzip independientes. Un desplazamiento virtual combina el byte c donde empieza un bloque comprimido con la posición u dentro del bloque descomprimido. Abajo: los primeros niveles de la jerarquía de casillas (dibujados a escala solo en su número relativo). La lectura B cabe dentro de una casilla de 1 Mb y se asigna a ella; la lectura A cruza la frontera entre dos casillas de nivel 3 y sube a la casilla de 8 Mb que las contiene. Consultar una región implica leer unas pocas casillas por nivel, nunca el archivo completo.

DONDE	
l	Nivel de la jerarquía: 0 es una casilla de 512 Mb; 5 son casillas de 16 kb.
2^{29-3l}	Tamaño de cada casilla del nivel l ($2^{29} \approx 5,4 \times 10^8$ pb en el nivel 0).
$\text{off}(l)$	Número de la primera casilla del nivel l (suma de las casillas de los niveles anteriores).
$[s, e)$	Intervalo o-based semiabierto que ocupa el alineamiento.

Para cada casilla, el índice guarda los intervalos de desplazamientos virtuales (*chunks*) donde se encuentran sus alineamientos. Para consultar una región se calculan las casillas que la solapan (a lo sumo una por nivel más las vecinas; unas pocas decenas en total), se leen solo los bloques correspondientes y se descartan los alineamientos que no solapan de verdad. Un índice lineal adicional, con una entrada por ventana de 16 kb, permite saltarse los *chunks* de casillas grandes que terminan antes de la región. El formato tabix generaliza exactamente este mecanismo a cualquier archivo de texto tabular ordenado y comprimido con BGZF, como VCF, BED o GFF (Li, 2011).

Ejemplo 2.5. (¿En qué casilla cae *TP53*?)

El transcrito canónico de *TP53* ocupa [7 668 420, 7 687 490) del cromosoma 17. En el nivel 5 sus extremos caen en casillas distintas ($\lfloor s/2^{14} \rfloor = 468$, $\lfloor (e - 1)/2^{14} \rfloor = 469$); en el nivel 4 ambos dan 58, así que su casilla es $\text{off}(4) + 58 = 585 + 58 = 643$. Una lectura de 150 nt en [7 674 219, 7 674 369) cabe en la casilla de nivel 5 número $4681 + 468 = 5149$.

La última vuelta de tuerca es CRAM (Hsi-Yang Fritz et al., 2011). Su idea es que la inmensa mayoría de las bases de una lectura coinciden con la referencia, así que basta guardar la posición y las *diferencias*: una lectura perfecta de 150 nt se reduce a unos pocos números. Combinado con modelos estadísticos para las calidades y los nombres, CRAM ocupa típicamente entre un 40 y un 60 % menos que BAM; a cambio, descomprimirlo exige disponer *exactamente* de la misma referencia con la que se creó (de ahí la importancia de versionar los ensamblajes, sección 2.2). HTSlib lee y escribe SAM, BAM y CRAM de forma transparente (Bonfield et al., 2021), y *samtools* expone esas capacidades en la línea de órdenes (Danecek et al., 2021).

2.1.8 VCF: las diferencias con la referencia

Si el BAM contiene todas las lecturas, el *Variant Call Format* (Danecek et al., 2011) contiene solo sus conclusiones: las posiciones donde una o más muestras difieren de la referencia. Un VCF tiene metadatos (##) que declaran cada campo, una línea de encabezado (#CHROM) y una fila por variante con ocho columnas fijas (CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO), seguidas de una columna FORMAT y una columna por muestra.

variantes.vcf

```
##fileformat=VCFv4.2
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotipo">
##FORMAT=<ID=PL,Number=G,Type=Integer,Description="Verosimilitudes">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT S1 S2
chr17 101 . C T 50 PASS AC=3 GT:PL 0/1:50,0,90 1/1:.
chr17 102 . CAG C 38 PASS AC=1 GT:PL 0/1:. 0/0:.
```

POS es 1-based. El genotipo GT de un diploide lista dos alelos (0 = REF, 1 = primer ALT, 2 = segundo ALT...): 0/1 es heterocigoto y 1/1 homocigoto alternativo; con

| en lugar de / el genotipo está *fasado*, es decir, se sabe qué alelo está en cada cromosoma homólogo.

Verosimilitudes de genotipo

El campo PL resume la evidencia que llevó al genotipo, y es otra aplicación de la escala Phred. Supongamos que en una posición cubierta por n lecturas observamos a con el alelo alternativo y $n - a$ con el de referencia, y que cada base se lee mal con probabilidad ε (su calidad Phred). Bajo cada genotipo diploide G , cada lectura muestrea uno de los dos cromosomas al azar, de modo que

$$P(D | G) = \prod_{j=1}^n \left[\frac{1}{2} P(d_j | A_1) + \frac{1}{2} P(d_j | A_2) \right], \quad PL_G = -10 \log_{10} \frac{P(D | G)}{\max_{G'} P(D | G')}, \quad (2.17)$$

DONDE

$D = (d_1, \dots, d_n)$ Bases observadas en las n lecturas que cubren la posición.

$G = A_1/A_2$ Genotipo diploide candidato (o/o, o/1 o 1/1).

$P(d_j | A)$ Probabilidad de leer d_j si el alelo verdadero es A : $1 - \varepsilon$ si coinciden, ε si no.

PL_G Verosimilitud del genotipo en escala Phred, normalizada para que el más verosímil valga 0.

Ejemplo 2.6. (Del recuento de lecturas al campo PL)

Diez lecturas cubren la posición 101: seis dicen **C** (REF) y cuatro **T** (ALT), con $\varepsilon = 0,01$ ($Q = 20$). Entonces

$$P(D | 0/0) = 0,99^6 0,01^4 = 9,41 \times 10^{-9},$$

$$P(D | 0/1) = 0,5^{10} = 9,77 \times 10^{-4},$$

$$P(D | 1/1) = 0,01^6 0,99^4 = 9,61 \times 10^{-13}.$$

El heterocigoto es el más verosímil ($PL = 0$); para los otros, $-10 \log_{10}(9,41 \times 10^{-9}/9,77 \times 10^{-4}) = 50$ y $-10 \log_{10}(9,61 \times 10^{-13}/9,77 \times 10^{-4}) = 90$. El VCF guarda $PL=50, 0, 90$: el homocigoto de referencia es 10^5 veces menos verosímil que el heterocigoto.

A partir de los genotipos de N individuos diploides se estima la frecuencia del alelo alternativo, $\widehat{AF} = \sum_i g_i / (2N)$, donde $g_i \in \{0, 1, 2\}$ es el número de copias del alelo alternativo del individuo i ; los campos AC y AN de INFO guardan precisamente

el numerador y el denominador. Volveremos a esta cantidad al estudiar genética de poblaciones.

Una variante, muchas escrituras

Las inserciones y deleciones se escriben con una *base ancla*: la base anterior al evento, para que ni REF ni ALT queden vacíos. Pero en regiones repetitivas la misma variante admite varias escrituras. Si la referencia es **GCAGAGT** (posiciones 101 a 107) y la muestra perdió una copia de **AG**, tanto POS=102 REF=CAG ALT=C como POS=104 REF=GAG ALT=G producen la misma secuencia **GCAGT**. Dos VCF que describen la misma variante de maneras distintas no “casan” al compararlos. Tan et al. (2015) demostraron que toda variante tiene una única representación *normalizada*: la más parsimoniosa (sin bases redundantes a ambos lados) y alineada a la izquierda, en este caso la de POS=102. Normalizar (`bcftools norm`) antes de comparar o anotar variantes es una práctica obligatoria.

2.2 Bases de datos biológicas desde Python

 Colab · 2.2 · Bases de datos

 abrir notebook

Un sistema de bibliotecas

Una ciudad con buenas bibliotecas no guarda cada libro en un solo edificio. Hay un archivo nacional que conserva, sin tocarlos, todos los manuscritos depositados; hay bibliotecas especializadas cuyos bibliotecarios escogen la mejor edición de cada obra y le añaden notas; y hay catálogos que no guardan libros, sino fichas que dicen dónde está cada uno y qué otros libros lo citan. Todo el sistema funciona porque cada libro tiene un número de catálogo que nunca cambia, y porque las nuevas ediciones reciben un número de edición en lugar de borrar la anterior. Las bases de datos biológicas están organizadas exactamente así.

2.2.1 Primarias, curadas e integradoras

Cada día, laboratorios de todo el mundo producen secuencias, estructuras y mediciones. Desde los años ochenta, la comunidad acordó depositarlas en repositorios públicos, gratuitos y sincronizados. Conviene distinguir tres niveles (figura 2.10):

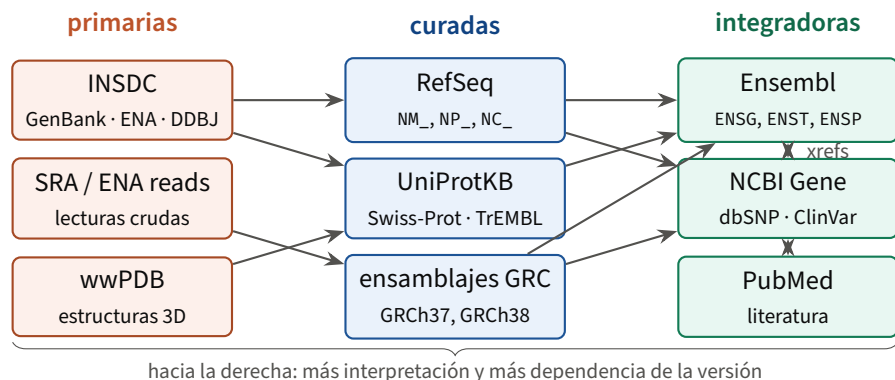


Figura 2.10. El ecosistema de bases de datos organizado por nivel de interpretación. Las flechas indican de dónde toma cada recurso sus datos; las líneas discontinuas, referencias cruzadas (*xrefs*) mantenidas entre recursos. Los archivos primarios nunca reescriben lo depositado; las bases curadas eligen y anotan una secuencia por entidad; los recursos integradores la sitúan en un genoma y la conectan con variantes, literatura y otras especies. Un mismo gen tiene un identificador distinto en cada caja.

- **Archivos primarios:** guardan lo que los autores depositaron, tal cual, con su número de acceso. Son GenBank, ENA y DDBJ para secuencias (Arita et al., 2021), el SRA para lecturas crudas (Leinonen et al., 2011) y el *Protein Data Bank* para estructuras. Nadie “corrige” un registro ajeno: si hay diez depósitos de la misma proteína, hay diez registros.
- **Bases curadas:** expertos (o procesos automáticos con reglas explícitas) eligen una secuencia representativa por entidad biológica y la anotan con evidencia. Es el caso de RefSeq, del NCBI (O’Leary et al., 2016), y de UniProtKB/Swiss-Prot (The UniProt Consortium, 2025).
- **Recursos integradores:** combinan ensamblajes, anotaciones, variantes y comparaciones entre especies, y enlazan con todo lo demás. Ensembl (Dyer et al., 2025) y el conjunto de bases del NCBI (Gene, dbSNP, ClinVar, PubMed...) (Sayers, Beck et al., 2025) son los ejemplos más usados.

2.2.2 Identificadores estables y versiones

Si hay una lección que llevarse de esta sección es esta: *un nombre de gen no es un identificador*. “*TP53*” designa, según el contexto, un locus del cromosoma 17, cualquiera de sus decenas de transcritos, varias isoformas proteicas, el gen ortólogo

de otra especie o la proteína p53 en general. Los identificadores de las bases de datos, en cambio, tienen dos propiedades que los hacen útiles:

Definición 2.4. (Identificador estable versionado)

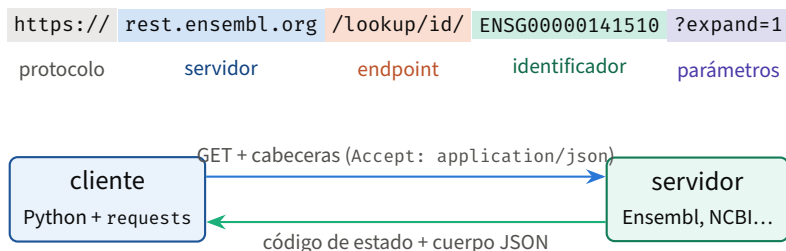
Un identificador estable es una cadena que designa siempre la misma entidad, que nunca se reasigna a otra y que se sigue resolviendo aunque la entidad quede obsoleta. Cuando el contenido de la entidad cambia, el identificador conserva su parte estable y se incrementa un número de versión: `acceso.versión`.

Así, NM_000546 es el ARNm de referencia de *TP53* en RefSeq, y NM_000546.6 es su sexta versión; cualquier cambio en la secuencia, por pequeño que sea, genera la .7. Ensembl sigue la misma lógica (ENST00000269305.9). UniProt separa dos contadores: la entrada P04637 va por la versión 315 de su *anotación*, pero por la versión 4 de su *secuencia*, que no cambia desde el 24 de noviembre de 2009. En un artículo o en un cuaderno de laboratorio hay que registrar el identificador *con* su versión, y la versión de la base de datos consultada.

Los identificadores tienen además una sintaxis reconocible: ENSG, ENST y ENSP seguidos de once dígitos en Ensembl; NM_, NP_ o NC_ en RefSeq; seis o diez caracteres alfanuméricos en UniProt; cuatro caracteres en el PDB. Esa regularidad permite validarlos con expresiones regulares antes de enviarlos a un servidor. Los de *TP53* se reúnen en la figura 2.15.

El ensamblaje también tiene versión

Las coordenadas genómicas solo tienen sentido respecto de un ensamblaje. Entre GRCh37 (2009) y GRCh38 (2013) se corrigieron errores, se cerraron huecos y se añadieron secuencias, de modo que casi todas las coordenadas cambiaron. *TP53* está en 17:7565097-7590856 en GRCh37 y en 17:7661779-7687546 en GRCh38, unos 96 682 pb más adelante; su identificador de gen es el mismo, pero su versión pasó de 11 a 21, y la del transcrito canónico, de 4 a 9. Mezclar un VCF en GRCh37 con una anotación en GRCh38 no produce ningún error de ejecución: produce resultados silenciosamente equivocados. La traslación de coordenadas entre ensamblajes (*liftover*) es posible, pero no es trivial, porque algunas regiones no tienen equivalente.



200 OK: leer el cuerpo. **400/404** pregunta mal formada o recurso inexistente: no reintentar. **429** demasiadas peticiones: esperar (Retry-After) y reintentar. **5xx** fallo del servidor: reintentar con espera creciente.

Figura 2.11. Arriba: anatomía de una URL de la API REST de Ensembl (Yates et al., 2015), que pide el gen *TP53* con sus transcritos y exones (`expand=1`). Abajo: el ciclo petición-respuesta y la interpretación de los códigos de estado más frecuentes. La distinción importante es entre errores del cliente (4xx: repetir la petición no servirá, salvo el 429) y errores transitorios (429 y 5xx: vale la pena esperar y reintentar).

📖 De un libro de proteínas a millones de secuencias

El primer “banco de datos” de secuencias fue un libro: el *Atlas of Protein Sequence and Structure* que Margaret Dayhoff publicó desde 1965 y que, en el ??, dio origen a las matrices PAM. GenBank se fundó en 1982 en Los Álamos y pasó al NCBI a finales de esa década; el *Protein Data Bank* se creó en 1971 con apenas siete estructuras (Berman et al., 2000). En 2003 los centros que lo gestionaban en Estados Unidos, Europa y Japón formalizaron el *worldwide PDB*, un único archivo global mantenido en colaboración (Berman et al., 2003), siguiendo el modelo que la INSDC ya aplicaba a las secuencias.

2.2.3 APIs REST: preguntarle a un servidor

Casi todas las bases de datos modernas ofrecen, además de su página web, una *API REST*: una forma de consultar sus datos mediante direcciones web que devuelven texto estructurado (JSON, XML, FASTA) en lugar de páginas para humanos. Una petición tiene cuatro partes (figura 2.11): el protocolo y el servidor, el *endpoint* (qué tipo de pregunta hacemos), el recurso (sobre qué entidad) y los parámetros (cómo queremos la respuesta). El servidor responde con un *código de estado* de tres dígitos y un cuerpo.

La respuesta JSON se convierte directamente en diccionarios y listas de Python. En el caso de Ensembl, un gen contiene una lista `Transcript`; cada transcrito, una lista `Exon` y, si codifica proteína, un diccionario `Translation`.

Límites de tasa

Los servidores públicos atienden a millones de usuarios y protegen su capacidad con *límites de tasa*. Las E-utilities del NCBI admiten 3 peticiones por segundo sin registro y 10 con una clave de API gratuita; Ensembl admite en promedio unas 15 por segundo. Quien los excede recibe respuestas 429 y, si insiste, puede ver bloqueada su dirección IP (y la de todo su laboratorio, que comparte la misma).

Un límite de r peticiones por segundo impone una cota inferior al tiempo de una descarga de N peticiones:

$$T \geq \frac{N}{r}. \quad (2.18)$$

DONDE

T	Tiempo total de la descarga, en segundos.
N	Número de peticiones necesarias.
r	Tasa máxima admitida por el servidor (peticiones por segundo).

La consecuencia práctica es que el único modo de acelerar una descarga grande es reducir N : pedir muchos registros por petición. Las E-utilities lo permiten a través de su *History server*: `esearch` con `usehistory=y` guarda la lista de resultados en el servidor y devuelve dos claves (`WebEnv` y `query_key`), con las que `efetch` descarga los registros en lotes de hasta varios cientos.

Ejemplo 2.7. (Descargar 5000 proteínas)

Queremos las secuencias FASTA de 5000 proteínas del NCBI. Pidiéndolas de una en una ($N = 5000$) sin clave de API ($r = 3$), la ecuación (2.18) da $T \geq 1667$ s, casi media hora; con clave ($r = 10$), 500 s. Si en cambio usamos el *History server* y lotes de 500 registros, bastan una petición `esearch` y $\lceil 5000/500 \rceil = 10$ peticiones `efetch`: $N = 11$ y $T \geq 3,7$ s. El cuello de botella deja de ser el límite de tasa y pasa a ser el volumen de datos transferido.

Reintentos con espera exponencial

Los errores transitorios (429, 5xx, cortes de red) se tratan reintentando, pero no inmediatamente: si mil clientes reintentan a la vez tras una caída, la provocan de nuevo. La estrategia estándar es la *espera exponencial con variación aleatoria*:

$$t_k = \min(t_{\text{máx}}, t_0 2^k) + U_k, \quad U_k \sim \text{Uniforme}(0, t_0), \quad (2.19)$$

DONDE

t_k	Tiempo de espera antes del reintentó k ($k = 0, 1, 2, \dots$).
t_0	Espera inicial (típicamente 1 s o el valor de la cabecera Retry-After).
$t_{\text{máx}}$	Espera máxima, para no esperar indefinidamente.
U_k	Variación aleatoria (<i>jitter</i>) que desincroniza a los clientes.

Con $t_0 = 1$ s y cinco reintentos, la espera determinista total es $1 + 2 + 4 + 8 + 16 = 31$ s. Si cada intento falla de forma independiente con probabilidad $q = 0,2$, la probabilidad de que fallen los seis (el original y los cinco reintentos) es $q^6 = 6,4 \times 10^{-5}$: la espera exponencial convierte un servicio poco fiable en uno casi perfecto a cambio de latencia ocasional.

Un cliente robusto combina estas ideas con una *caché* (no preguntar dos veces lo mismo) y un *registro de procedencia* (qué se pidió, a quién, cuándo, y si la respuesta vino del servidor o de la caché). El notebook de esta lección construye la función `fetch()` que usaremos en el resto del curso; su núcleo es el siguiente:

```
cliente.py

import time, random, requests

def get_json(url, params=None, t0=1.0, tmax=30, tries=6):
    for k in range(tries):
        r = requests.get(url, params=params, timeout=30,
                        headers={"Accept": "application/json"})
        if r.status_code == 200:
            return r.json()
        if r.status_code not in (429, 500, 502, 503, 504):
            r.raise_for_status() # error del cliente
        wait = float(r.headers.get("Retry-After", 0))
        wait = max(wait, min(tmax, t0 * 2 ** k))
        time.sleep(wait + random.uniform(0, t0))
    raise RuntimeError(f"sin respuesta tras {tries} intentos")
```

2.2.4 NCBI y las E-utilities

El NCBI agrupa más de treinta bases de datos (Nucleotide, Protein, Gene, PubMed, SRA, dbSNP, ClinVar, Taxonomy...) bajo un sistema de búsqueda común llamado Entrez (Sayers, Beck et al., 2025). Sus *Entrez Programming Utilities* son una familia de servicios que se combinan como piezas:

Utilidad	Pregunta que responde	Devuelve
esearch	¿qué registros cumplen esta búsqueda?	identificadores (UID) y recuento
esummary	¿qué dice el resumen de estos registros?	metadatos
elink	¿qué registros de otra base están ligados a estos?	UID relacionados
efetch	dame el registro completo	FASTA, GenBank, XML...

El flujo típico es `esearch` → `esummary/elink` → `efetch`: primero se encuentra, luego se mira, al final se descarga. Para *TP53*, `esearch` en la base Gene con la consulta `TP53[sym] AND human[orgn]` devuelve el UID 7157; `esummary` devuelve su localización (ejemplo 2.2); `elink` con `linkname=gene_protein_refseq` lleva a sus proteínas RefSeq, y `efetch` descarga NP_000537.3 en FASTA, una proteína de 393 aminoácidos. Biopython envuelve estas llamadas en el módulo `Bio.Entrez`, que además respeta automáticamente el límite de tasa (Cock et al., 2009).

```
entrez_tp53.py

from Bio import Entrez, SeqIO

Entrez.email = "usted@universidad.edu"          # obligatorio
# Entrez.api_key = "..."                       # opcional: 10/s
h = Entrez.esearch(db="gene", term="TP53[sym] AND human[orgn]")
gene_id = Entrez.read(h)["IdList"][0]           # '7157'
h = Entrez.elink(dbfrom="gene", db="protein", id=gene_id,
                 linkname="gene_protein_refseq")
links = Entrez.read(h)[0]["LinkSetDb"][0]["Link"]
h = Entrez.efetch(db="protein", id="NP_000537.3",
                  rettype="fasta", retmode="text")
p53 = SeqIO.read(h, "fasta")
print(gene_id, len(links), p53.id, len(p53))
```

Con el parámetro `rettype=count`, `esearch` devuelve solo el número de registros que cumplen una búsqueda, lo que permite medir, por ejemplo, cuántos artículos por año indexa PubMed sobre un tema; el notebook de la lección usa esa consulta

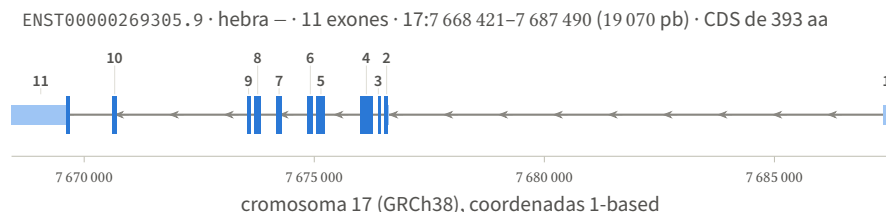


Figura 2.12. Modelo del transcrito canónico de *TP53* dibujado directamente a partir de la respuesta de la API de Ensembl. Los bloques gruesos son región codificante (CDS), los delgados son regiones no traducidas (UTR) y la línea con flechas son intrones; el gen se transcribe de derecha a izquierda porque está en la hebra negativa, así que el exón 1 es el de la derecha. El exón 1 es completamente no codificante: la proteína empieza en el exón 2. Los 11 exones suman 2512 nt, apenas un 13 % de los 19 070 pb del transcrito; el resto son intrones.

para estimar el tiempo de duplicación de la literatura sobre CRISPR (menos de dos años entre 2013 y 2020).

2.2.5 Ensembl: el gen en su genoma

Ensembl anota automáticamente los genomas de cientos de especies de vertebrados y, en su versión actual, integra además los resultados del proyecto GENCODE para el ser humano y el ratón (Dyer et al., 2025). Su API REST (Yates et al., 2015) expone casi todo su contenido; el endpoint `lookup/id` con `expand=1` devuelve el gen con todos sus transcritos y exones.

Para *TP53* en GRCh38, Ensembl informa un gen de 25 768 pb en la hebra negativa del cromosoma 17, con 40 transcritos anotados: distintos sitios de inicio, combinaciones de exones por *splicing* alternativo y transcritos que no codifican proteína. Entre ellos, uno se marca como *canónico*: ENST00000269305.9, que corresponde al ARNm de referencia NM_000546.6 de RefSeq. La figura 2.12 lo dibuja a partir de la respuesta de la API.

⚠ «La proteína de *TP53*» no existe

Con 40 transcritos anotados, la frase “la secuencia de *TP53*” es ambigua. Hay que especificar el transcrito (ENST00000269305.9 o NM_000546.6) y el ensamblaje. Las herramientas de anotación de variantes pueden describir la misma mutación con posiciones proteicas distintas según el transcrito

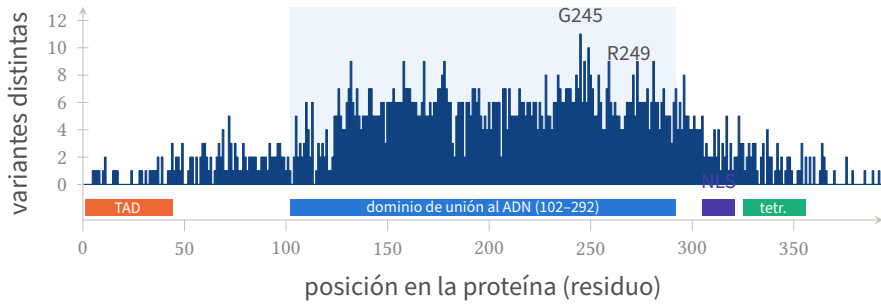


Figura 2.13. Número de variantes de un aminoácido distintas registradas en UniProtKB (P04637) en cada posición de p53. Abajo, las regiones funcionales anotadas en la misma entrada: dominio de transactivación (TAD, 1–44), dominio de unión al ADN (102–292), señal de localización nuclear bipartita (NLS) y dominio de oligomerización o tetramerización (325–356). Las variantes se concentran de forma abrumadora en el dominio de unión al ADN (fondo azul), donde están las posiciones con más variantes distintas, como G245 y R249.

elegido; por eso las recomendaciones clínicas exigen el transcrito de referencia versionado.

2.2.6 UniProt: la proteína y su conocimiento

UniProt es el recurso central para las proteínas (The UniProt Consortium, 2025). Su base de conocimiento, UniProtKB, tiene dos secciones: Swiss-Prot, revisada manualmente por curadores que resumen la literatura con evidencia explícita, y TrEMBL, anotada automáticamente y órdenes de magnitud mayor. La entrada revisada P04637 (p53 humana, 393 aminoácidos) describe sus dominios, sitios de modificación, interacciones, isoformas, estructuras y más de mil variantes naturales, casi todas mutaciones somáticas halladas en tumores. La API REST las devuelve en JSON desde <https://rest.uniprot.org/uniprotkb/P04637.json>, como una lista de features con su tipo y localización.

La figura 2.13 sugiere que las variantes no están repartidas al azar. Podemos cuantificarlo con una prueba binomial, un buen ejemplo de cómo los datos de una base de datos se convierten en una afirmación estadística.

Ejemplo 2.8. (¿Se concentran las variantes en el dominio de unión al ADN?)

La entrada registra $N = 1363$ variantes naturales en 332 posiciones distintas. El dominio de unión al ADN abarca 191 de los 393 residuos, una fracción $\pi_0 = 191/393 = 0,486$. Si las variantes cayeran al azar a lo largo de la proteína, el número dentro del dominio, K , seguiría una distribución $\text{Bin}(N, \pi_0)$, con media $N\pi_0 = 662,4$ y desviación típica $\sqrt{N\pi_0(1 - \pi_0)} = 18,5$. Se observan $K = 1052$ (el 77 %), lo que da

$$z = \frac{1052 - 662,4}{18,5} = 21,1,$$

y una probabilidad exacta $\mathbb{P}(K \geq 1052) \approx 8 \times 10^{-104}$. La concentración es incuestionable. Pero cuidado con la interpretación: el resultado mezcla biología (las mutaciones que destruyen la unión al ADN son las seleccionadas en los tumores) con un *sesgo de verificación*, porque durante años muchos estudios secuenciaron solo los exones 5 a 8, que codifican ese dominio. Los datos de una base de datos reflejan también cómo se recolectaron.

2.2.7 PDB: la estructura

El *Protein Data Bank* archiva las estructuras tridimensionales de macromoléculas determinadas experimentalmente (Berman et al., 2000). Cada entrada tiene un identificador de cuatro caracteres, las coordenadas atómicas (en los formatos PDB clásico o mmCIF) y los metadatos del experimento: método, resolución, cadenas, ligandos. El portal estadounidense, RCSB PDB, ofrece además APIs de búsqueda y de datos (esta última en GraphQL), y desde 2022 muestra, junto a las estructuras experimentales, modelos computacionales predichos por inteligencia artificial, claramente separados (Burley et al., 2025).

Una búsqueda de todas las entradas cuyas cadenas remiten al acceso UniProt Po4637 devuelve 323 estructuras de p53 humana. La figura 2.14 resume su historia: 275 por difracción de rayos X, 30 por resonancia magnética nuclear y 16 por criomicroscopía electrónica. La resolución mediana es de 1,8 Å para rayos X y de 3,83 Å para criomicroscopía; la mejor alcanza 1,01 Å.

La entrada 1TSR (publicada en el PDB en enero de 1996, 2,2 Å) contiene tres copias del dominio de unión al ADN de p53 (cadenas A, B y C) unidas a una doble hélice de ADN (cadenas E y F). Su publicación (Cho et al., 1994) fue un hito, porque mostró por qué los “puntos calientes” de mutación en cáncer son los que son: residuos como R248 y R273 contactan directamente el ADN, y otros como R175, G245 y R249 sostienen la superficie de contacto. Una sola sustitución en esos lugares basta para

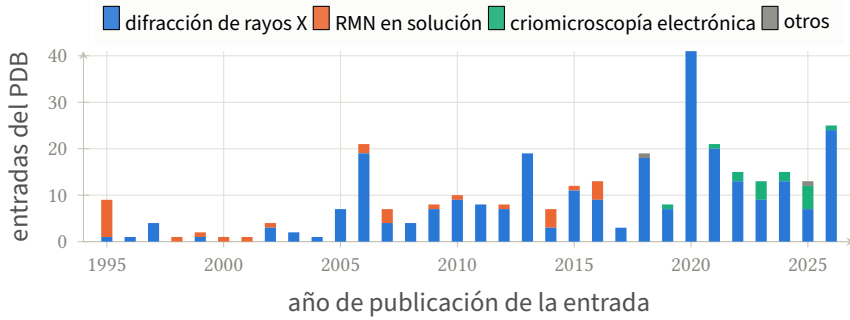


Figura 2.14. Entradas del PDB que contienen p53 humana (acceso UniProt P04637), por año de publicación y método experimental, obtenidas de la API de RCSB PDB en septiembre de 2026 (el último año está incompleto). Las primeras estructuras de mediados de los noventa, como 1TSR, resolvieron el dominio de unión al ADN; la mayoría de las posteriores son variantes de ese dominio con fármacos candidatos a “rescatar” la p53 mutada. La criomicroscopía aparece solo en los últimos años, típicamente con complejos grandes y resoluciones más modestas.

que p53 suelte el ADN o pierda su forma. Acabamos de encadenar tres bases de datos (la variante en UniProt, la posición en el genoma de Ensembl y la estructura en el PDB) para explicar un fenotipo.

2.2.8 El pasaporte de un gen y los principios FAIR

Los recursos que hemos visitado no son islas: cada uno mantiene referencias cruzadas hacia los demás. El endpoint `xrefs/id` de Ensembl, por ejemplo, devuelve para ENSG00000141510 su identificador HGNC, su UID de NCBI Gene y su número OMIM, y para la proteína ENSP00000269305 su acceso de UniProt, su proteína RefSeq y la lista de estructuras del PDB. Con esas referencias se construye el “pasaporte” de la figura 2.15.

Esta capacidad de pasar de un recurso a otro sin intervención humana no es casual. Wilkinson et al. (2016) la formularon como cuatro principios que deberían cumplir todos los datos científicos, resumidos en el acrónimo FAIR:

- **F** (*Findable*, localizables): los datos tienen identificadores globales y persistentes, y metadatos ricos indexados en un recurso consultable.
- **A** (*Accessible*, accesibles): se recuperan por su identificador mediante un protocolo abierto y estandarizado (como HTTP), y los metadatos siguen disponibles aunque los datos dejen de estarlo.

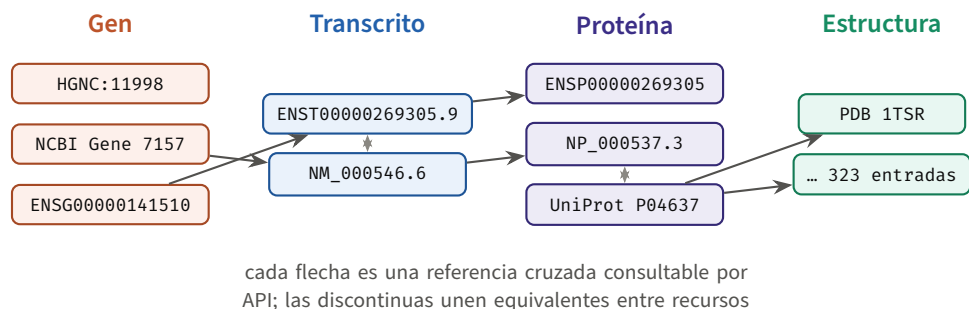


Figura 2.15. El “pasaporte” de *TP53*, construido con los identificadores obtenidos en esta sección. El nivel de detalle crece de izquierda a derecha, como en el dogma central: un gen, uno de sus transcritos, la proteína que codifica y una de sus estructuras. Cada identificador de transcrito y proteína lleva su versión; los de gen y estructura designan entidades cuyo contenido puede actualizarse sin cambiar de nombre, por lo que conviene registrar también la versión de la base de datos consultada.

- **I** (*Interoperable*, interoperables): usan lenguajes y vocabularios formales y compartidos (como la *Sequence Ontology*) e incluyen referencias a otros datos.
- **R** (*Reusable*, reutilizables): tienen una licencia clara, una procedencia detallada y cumplen los estándares de su comunidad.

Todo lo que hemos visto en este capítulo es una implementación concreta de esos principios: identificadores versionados (F), APIs REST (A), formatos y ontologías estándar (I). El cuarto principio, la reutilización, depende de nosotros: de que registremos *qué* pedimos, *a quién*, *cuándo* y en *qué versión*.

Identificador + versión + fecha + versión de la base.
Sin esos cuatro datos, un análisis no es reproducible.

☰ En resumen

- FASTA guarda secuencias; FASTQ añade una calidad Phred por base, $Q = -10 \log_{10} p$, codificada como $\text{chr}(Q+33)$. Los errores esperados, $EE = \sum 10^{-Q_i/10}$, resumen una lectura mejor que su calidad media.

- GenBank, GFF3, GTF y SAM/VCF usan coordenadas 1-based cerradas; BED, BAM y Python, 0-based semiabiertas. La conversión es $s_0 = s_1 - 1$, $e_0 = e_1$, y el REF de un VCF es la prueba de fuego.
- En SAM, la CIGAR determina la longitud de la lectura y el tramo de referencia según qué operaciones consumen cada secuencia; la FLAG codifica doce condiciones como suma de potencias de 2; MAPQ es la calidad Phred de la posición.
- BAM y CRAM comprimen en bloques BGZF; un índice jerárquico de 37 449 casillas y desplazamientos virtuales $v = c \cdot 2^{16} + u$ permite leer una región sin descomprimir todo el archivo. VCF describe variantes con genotipos y verosimilitudes (PL) en escala Phred, y debe normalizarse.
- Las bases de datos se organizan en primarias (INSDC, SRA, PDB), curadas (RefSeq, Swiss-Prot) e integradoras (Ensembl, NCBI Gene). Se consultan mediante APIs REST, respetando límites de tasa ($T \geq N/r$), reintentando con espera exponencial y guardando caché y procedencia.
- Un gen se identifica con identificadores estables *versionados* y en un ensamblaje concreto; las referencias cruzadas entre recursos materializan los principios FAIR.

Ejercicios

Dificultad:    básica    intermedia    avanzada

- 2.1**    (a) ¿Qué calidad Phred corresponde a una probabilidad de error de 0,0005? (b) ¿Con qué carácter se escribe en Phred+33? (c) Decodifique la cadena 5?I+ a calidades y probabilidades de error.
- 2.2**    Descomponga las FLAG 83, 2064 y 3844. ¿Qué filtro -F de `samtools view` excluye a la vez lecturas no alineadas, secundarias, duplicadas y suplementarias?
- 2.3**    Para la CIGAR 3S40M1I20M5D30M2S con POS=1000: (a) ¿qué posición de la referencia queda frente a la base 50 de la lectura? (b) ¿Qué intervalo 0-based semiabierto devuelve `reference_start`, `reference_end` en `pysam`? Escriba una función que, dada una CIGAR y POS, devuelva el par de listas de posiciones alineadas.

- 2.4 ●●● Usando la ecuación (2.6), calcule la calidad Phred uniforme mínima que debe tener una lectura de 150 nt para que la probabilidad de estar libre de errores sea al menos 0,9. Compare la aproximación de Poisson con el cálculo exacto.
- 2.5 ●●● Implemente la función `reg2bin(beg, end)` del teorema 2.2 y verifique los resultados del ejemplo 2.5. ¿Cuántas casillas hay que examinar como máximo para una región de 100 kb?
- 2.6 ●●● Escriba un cliente que obtenga para *BRCA1* humano su UID de NCBI Gene (`esearch`), su identificador de Ensembl (`endpoint lookup/symbol`), su transcrito canónico y su acceso de UniProt (`xrefs`), respetando el límite de tasa de cada servicio y registrando la fecha y la versión de cada respuesta.
- 2.7 ●●● Repita el ejemplo 2.6 para 20 lecturas con 3 alternativas y $\varepsilon = 0,01$, y luego con $\varepsilon = 0,05$. ¿Cómo cambia la decisión entre o/o y o/1? Explique por qué una calidad mal calibrada (figura 2.5) produce falsos heterocigotos.

Referencias del capítulo

- Arita, M., Karsch-Mizrachi, I., & Cochrane, G. (2021). The international nucleotide sequence database collaboration. *Nucleic Acids Research*, 49(D1), D121-D124. <https://doi.org/10.1093/nar/gkaa967>
- Berman, H., Henrick, K., & Nakamura, H. (2003). Announcing the worldwide Protein Data Bank. *Nature Structural & Molecular Biology*, 10(12), 980. <https://doi.org/10.1038/nsb1203-980>
- Berman, H. M., Westbrook, J., Feng, Z., Gilliland, G., Bhat, T. N., Weissig, H., Shindyalov, I. N., & Bourne, P. E. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28(1), 235-242. <https://doi.org/10.1093/nar/28.1.235>
- Bonfield, J. K., Marshall, J., Danecek, P., Li, H., Ohan, V., Whitwham, A., Keane, T., & Davies, R. M. (2021). HTSlib: C library for reading/writing high-throughput sequencing data. *GigaScience*, 10(2), giaboo7. <https://doi.org/10.1093/gigascience/giab007>
- Buffalo, V. (2015). *Bioinformatics data skills: Reproducible and robust research with open source tools*. O'Reilly Media.
- Burley, S. K., Bhatt, R., Bhikadiya, C., Bi, C., Biester, A., Biswas, P., Bittrich, S., Blaumann, S., Brown, R., Chao, H., et al. (2025). Updated resources for exploring experimentally-determined PDB structures and Computed Structure Models at the RCSB Protein Data Bank. *Nucleic Acids Research*, 53(D1), D564-D574. <https://doi.org/10.1093/nar/gkae1091>

- Cho, Y., Gorina, S., Jeffrey, P. D., & Pavletich, N. P. (1994). Crystal structure of a p53 tumor suppressor-DNA complex: Understanding tumorigenic mutations. *Science*, 265(5170), 346-355. <https://doi.org/10.1126/science.8023157>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., & Rice, P. M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6), 1767-1771. <https://doi.org/10.1093/nar/gkp1137>
- Danecek, P., Auton, A., Abecasis, G., Albers, C. A., Banks, E., DePristo, M. A., Handsaker, R. E., Lunter, G., Marth, G. T., Sherry, S. T., McVean, G., Durbin, R., & 1000 Genomes Project Analysis Group. (2011). The variant call format and VCFtools. *Bioinformatics*, 27(15), 2156-2158. <https://doi.org/10.1093/bioinformatics/btr330>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giaboo8. <https://doi.org/10.1093/gigascience/giab008>
- Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., Barrera-Enriquez, V. P., Becker, A., Bennett, R., Beracochea, M., Berry, A., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1), D948-D957. <https://doi.org/10.1093/nar/gkae1071>
- Eilbeck, K., Lewis, S. E., Mungall, C. J., Yandell, M., Stein, L., Durbin, R., & Ashburner, M. (2005). The Sequence Ontology: A tool for the unification of genome annotations. *Genome Biology*, 6(5), R44. <https://doi.org/10.1186/gb-2005-6-5-r44>
- Ewing, B., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. II. Error probabilities. *Genome Research*, 8(3), 186-194. <https://doi.org/10.1101/gr.8.3.186>
- Ewing, B., Hillier, L., Wendl, M. C., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. I. Accuracy assessment. *Genome Research*, 8(3), 175-185. <https://doi.org/10.1101/gr.8.3.175>
- Hsi-Yang Fritz, M., Leinonen, R., Cochrane, G., & Birney, E. (2011). Efficient storage of high throughput DNA sequencing data using reference-based compression. *Genome Research*, 21(5), 734-740. <https://doi.org/10.1101/gr.114819.110>
- Kent, W. J., Sugnet, C. W., Furey, T. S., Roskin, K. M., Pringle, T. H., Zahler, A. M., & Haussler, D. (2002). The human genome browser at UCSC. *Genome Research*, 12(6), 996-1006. <https://doi.org/10.1101/gr.229102>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database), D19-D21. <https://doi.org/10.1093/nar/gkq1019>
- Li, H. (2011). Tabix: Fast retrieval of sequence features from generic TAB-delimited files. *Bioinformatics*, 27(5), 718-719. <https://doi.org/10.1093/bioinformatics/btq671>

- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>
- Lipman, D. J., & Pearson, W. R. (1985). Rapid and sensitive protein similarity searches. *Science*, 227(4693), 1435-1441. <https://doi.org/10.1126/science.2983426>
- O'Leary, N. A., Wright, M. W., Brister, J. R., Ciuffo, S., Haddad, D., McVeigh, R., Rajput, B., Robertse, B., Smith-White, B., Ako-Adjei, D., et al. (2016). Reference sequence (RefSeq) database at NCBI: Current status, taxonomic expansion, and functional annotation. *Nucleic Acids Research*, 44(D1), D733-D745. <https://doi.org/10.1093/nar/gkv1189>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: A flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Sayers, E. W., Beck, J., Bolton, E. E., Brister, J. R., Chan, J., Connor, R., Feldgarden, M., Fine, A. M., Funk, K., Hoffman, J., Kannan, S., Kelly, C., Klimke, W., Kim, S., Lathrop, S., Marchler-Bauer, A., Murphy, T. D., et al. (2025). Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Research*, 53(D1), D20-D29. <https://doi.org/10.1093/nar/gkae979>
- Sayers, E. W., Cavanaugh, M., Frisse, L., Pruitt, K. D., Schneider, V. A., Underwood, B. A., Yankie, L., & Karsch-Mizrachi, I. (2025). GenBank 2025 update. *Nucleic Acids Research*, 53(D1), D56-D61. <https://doi.org/10.1093/nar/gkae1114>
- Tan, A., Abecasis, G. R., & Kang, H. M. (2015). Unified representation of genetic variants. *Bioinformatics*, 31(13), 2202-2204. <https://doi.org/10.1093/bioinformatics/btv112>
- The UniProt Consortium. (2025). UniProt: The Universal Protein Knowledgebase in 2025. *Nucleic Acids Research*, 53(D1), D609-D617. <https://doi.org/10.1093/nar/gkae1010>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yates, A., Beal, K., Keenan, S., McLaren, W., Pignatelli, M., Ritchie, G. R. S., Ruffier, M., Taylor, K., Vullo, A., & Flicek, P. (2015). The Ensembl REST API: Ensembl data for any language. *Bioinformatics*, 31(1), 143-145. <https://doi.org/10.1093/bioinformatics/btu613>